

## 3 JSX Avancé

*Maintenant que vous connaissez les props, nous allons voir comment rendre le JSX dynamique et puissant !*

### 3.1 Expressions JavaScript dans JSX

#### 3.1.1 Syntaxe des Accolades { }

En JSX, les accolades { } permettent d'insérer des **expressions JavaScript**.

Listing 8 – Expressions basiques

```
function Welcome({ name, age }) { // Props destructurées
  return (
    <div>
      <h1>Bonjour {name}</h1>
      <p>Vous avez {age} ans</p>
      <p>Dans 5 ans, vous aurez {age + 5} ans</p>
    </div>
  );
}

// Utilisation
<Welcome name="Ahmed" age={20} />
```

#### ★ Astuce

##### Expression vs Statement :

- Expression = retourne une valeur (2 + 2, name.toUpperCase())
- Statement = action sans retour (if, for, while)

#### 3.1.2 Expressions Valides

Listing 9 – Types d'expressions autorisées

```
function ProductCard({ product }) { // Props avec objet
  return (
    <div>
      /* Variables et propriétés */
      <h2>{product.name}</h2>

      /* Calculs */
      <p>Prix TTC: {product.price * 1.2} DH</p>

      /* Méthodes de chaînes */
      <p>Marque: {product.brand.toUpperCase()}</p>

      /* Appels de fonctions */
      <p>Date: {new Date().toLocaleDateString()}</p>

      /* Template literals */
      <p>`Produit: ${product.brand} ${product.name}`</p>
    </div>
  );
}
```

## △ Important

### Erreurs courantes à éviter :

```
// MAUVAIS : Objects bruts
<p>{product}</p> // Error: Objects are not valid as React child

// MAUVAIS : Statements
<div>{if (x > 5) { return "Grand" }}</div>

// BON : Propriétés d'objets
<p>{product.name}</p>

// BON : Opérateur ternaire (expression)
<div>{x > 5 ? "Grand" : "Petit"}</div>
```

## 3.2 Rendu Conditionnel

Le rendu conditionnel permet d'afficher différents éléments selon une condition.

### 3.2.1 Opérateur Ternaire

Syntaxe : `condition ? valeurSiVrai : valeurSiFaux`

Listing 10 – Ternaire simple

```
function ProductCard({ name, price, inStock }) { // Props
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>{price} DH</p>
      {inStock ? (
        <span className="badge green">En stock</span>
      ) : (
        <span className="badge red">Rupture</span>
      )}
    </div>
  );
}
```

Quand utiliser : Deux alternatives (A ou B)

### 3.2.2 Short-circuit AND (&&)

Syntaxe : `condition && <Element />`

Listing 11 – Operateur AND

```
function AdminPanel({ isAdmin }) { // Props boolean
  return (
    <div>
      <h1>Dashboard</h1>
      {isAdmin && <button>Parametres Admin</button>}
    </div>
  );
}
```

#### ★ Astuce

Attention avec les nombres :

```
function CartBadge({ count }) { // Props count
  // PROBLEME : Affiche "0" (falsy mais rendu)
  {count && <span>Articles: {count}</span>}

  // MIEUX : Condition explicite
  {count > 0 && <span>Articles: {count}</span>}
}
```

Quand utiliser : Affichage conditionnel simple (présent ou absent)

### 3.2.3 If/Else Avant Return

Pour des conditions complexes, utilisez if/else classique avant le return :

### Listing 12 – If/else avec variable

```
function ProductBadge({ stock }) { // Props
  let badge;

  if (stock > 10) {
    badge = <span className="badge green">En stock</span>;
  } else if (stock > 0) {
    badge = <span className="badge orange">Stock faible</span>;
  } else {
    badge = <span className="badge red">Rupture</span>;
  }

  return <div>{badge}</div>;
}
```

**Quand utiliser :** Plus de 2 conditions ou logique complexe

### 3.3 Listes et Affichage Dynamique

#### 3.3.1 Méthode map()

La méthode `map()` transforme un tableau en tableau de composants JSX.

Listing 13 – Affichage liste simple

```
function FruitList() {
  const fruits = ['Pomme', 'Banane', 'Orange'];

  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

#### 3.3.2 La Prop Key (Cruciale!)

##### △ Important

**Règle absolue :** Chaque élément d'une liste DOIT avoir une prop `key` unique.

**Pourquoi ?**

- React utilise `key` pour identifier les éléments
- Optimise le re-render (réconciliation)
- Évite les bugs de state

Listing 14 – Key avec ID unique (recommande)

```
function ProductList({ products }) { // Props avec array
  return (
    <div className="products">
      {products.map(product => (
        <ProductCard
          key={product.id} // Key obligatoire
          name={product.name}
          price={product.price}
          inStock={product.inStock}
        />
      ))}
    </div>
  );
}

// Utilisation
const myProducts = [
  { id: 1, name: 'Laptop', price: 5000, inStock: true },
  { id: 2, name: 'Mouse', price: 150, inStock: false }
];

<ProductList products={myProducts} />
```

### ★ Astuce

#### Bonnes pratiques key :

- Utiliser un ID unique (database ID, UUID)
- Éviter l'index si : ordre change, suppression, insertion
- Index OK si : liste statique, jamais réordonnée

### 3.3.3 Filtrage Avant map()

Combinez `filter()` et `map()` pour affichage conditionnel :

Listing 15 – Filtrer puis afficher

```
function AvailableProducts({ products }) { // Props array
  // Filtrer produits en stock
  const available = products.filter(p => p.inStock);

  return (
    <div>
      <h2>{available.length} produits disponibles</h2>
      {available.map(product => (
        <ProductCard
          key={product.id}
          name={product.name}
          price={product.price}
        />
      ))}
    </div>
  );
}
```

## PAUSE COURS - Exercice Papier 2 (JSX + Props)

**Durée :** 20 minutes

**Consigne :** Sur papier, écrire le code complet

**Partie 1 :** Créer composant `UserCard` qui reçoit :

- name (string)
- email (string)
- isAdmin (boolean)
- age (number)

**Afficher :**

- Nom en `<h3>`
- Email en `<p>`
- Badge "Admin" si `isAdmin=true` (utiliser `&&`)
- Message "Majeur" si `age >= 18`, sinon "Mineur" (utiliser ternaire)

**Partie 2 :** Créer composant `UserList` qui reçoit :

- users (array d'objets avec id, name, email, isAdmin, age)

**Afficher :**

- Liste de `UserCard` avec `map()`
- Key prop = `user.id`